

Conversación: Sketch Arduino OLED — Poema "Queja"

Exportado desde Claude — animaciones OLED, depuración de bitmaps y errores de compilación

Usuario

Holaa, necesito ayuda con algo, y es que tengo este código inicial: [se adjuntó el sketch original de Arduino con el poema "Queja" de Alfonsina Storni, control por potenciómetro, animación de llamas y centrado de texto en pantalla OLED]

Claude

¡Hola! Le eché un vistazo a tu código para el Arduino con la pantalla OLED y el poema de Alfonsina Storni — se ve bastante elaborado (control por potenciómetro, animación de llamas, centrado de texto, etc.).

Pero tu mensaje se cortó justo antes de contarme qué necesitas. ¿Qué es lo que quieres hacer con este sketch? Por ejemplo:

- ¿Agregar o modificar alguna funcionalidad?
- ¿Corregir un bug o comportamiento raro?
- ¿Optimizar algo (memoria, velocidad, etc.)?
- ¿Entender cómo funciona alguna parte específica?

Cuéntame qué tienes en mente y seguimos desde ahí.

Usuario

[se adjuntó animacionQueja.h con 14 frames de bitmap] y quiero añadir una animación luego del primer verso que es: "Alfonsina Storni" y está en este archivo .h, y no sé cómo hacerlo

Claude

¡Perfecto, ya tienes todo lo necesario! El archivo animacionQueja.h ya define framesQueja[], numFramesQueja y frameDelayQueja, y ya lo estás incluyendo en el sketch. Solo falta conectar esa animación al flujo del programa, siguiendo el mismo patrón que ya usas para framesLlamas.

La idea es la misma que con la animación de llamas: reservar una posición extra del potenciómetro para la animación, pero esta vez justo después del verso 0 ("Alfonsina Storni") en vez de después del verso 8.

1. Agregar una segunda posición de animación

Ahora tendrás dos paradas de animación en la escala del potenciómetro, así que hay que sumar 2 (no 1) a totalPosiciones:

```
// Antes:
const int posicionAnimacion = 9;
const int totalPosiciones = totalVersos + 1; // 14

// Ahora:
const int posicionAnimacionQueja = 1; // justo después del verso 0 ("Alfonsina Storni")
const int posicionAnimacionLlamas = 10; // después del verso 8 (se corrió +1 por la nueva animación)
const int totalPosiciones = totalVersos + 2; // 15
```

2. Nueva función para reproducir la animación de "queja"

Copia el mismo patrón de reproducirAnimacionUnaVez(), pero usando el arreglo de animacionQueja.h:

```
void reproducirAnimacionQuejaUnaVez() {
  for (int i = 0; i < numFramesQueja; i++) {
    display.clearDisplay();
    display.drawBitmap(0, 0, framesQueja[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
```

```

    display.display();
    delay(frameDelayQueja);
  }
}

```

3. Actualizar loop() para reconocer ambas animaciones

Como ahora hay dos posiciones especiales, y el resto de los versos se van corriendo según cuántas animaciones haya antes de esa posición, el cálculo del índice del verso cambia un poco:

```

void loop() {

  int valorPot = analogRead(POT_PIN);
  int posicionActual = map(valorPot, 0, 1023, 0, totalPosiciones - 1);

  if (posicionActual != posicionAnterior) {

    if (posicionActual == posicionAnimacionQueja) {
      reproducirAnimacionQuejaUnaVez();
    } else if (posicionActual == posicionAnimacionLlamas) {
      reproducirAnimacionUnaVez();
    } else {
      // Calculamos cuántas animaciones ya "pasaron" antes de esta posición,
      // para saber cuánto restarle y obtener el índice real del verso
      int indiceActual = posicionActual;

      if (posicionActual > posicionAnimacionQueja) indiceActual--;
      if (posicionActual > posicionAnimacionLlamas) indiceActual--;

      mostrarVerso(versosPantalla[indiceActual], palabraGrande[indiceActual]);
      Serial.println(misVersos[indiceActual]);
    }

    posicionAnterior = posicionActual;
  }

  delay(20);
}

```

Cómo queda la escala completa (0 a 14): 0 = Verso 0 "Alfonsina Storni" · 1 = Animación "Queja" · 2 = Verso 1 "Señor, mi queja es ésta," · 3–9 = Versos 2–8 · 10 = Animación "Llamas" · 11–14 = Versos 9–12.

No necesitas tocar nada más: mostrarVerso(), misVersos[], versosPantalla[], etc. quedan exactamente igual, solo cambia cómo se traduce la posición del potenciómetro al índice del verso.

¿Quieres que la animación de "queja" se reproduzca solo una vez al pasar por ahí (como ya está configurado) o prefieres que se pueda repetir si el usuario vuelve a esa posición?

Usuario

En donde ubico el segundo cambio?

Claude

El segundo cambio (la función reproducirAnimacionQuejaUnaVez()) lo puedes ubicar justo debajo de la función reproducirAnimacionUnaVez() que ya existe en tu sketch, ya que cumplen el mismo rol (reproducir una animación) y así quedan agrupadas.

Es decir, en tu archivo principal (el .ino), busca este bloque:

```

void reproducirAnimacionUnaVez() {
  for (int i = 0; i < numFramesLlamas; i++) {
    display.clearDisplay();
    display.drawBitmap(0, 0, framesLlamas[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
    display.display();
  }
}

```

```

        delay(frameDelayLlamas);
    }
}

```

Y justo después de su llave de cierre "}", pega la nueva función:

```

void reproducirAnimacionUnaVez() {
    for (int i = 0; i < numFramesLlamas; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesLlamas[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayLlamas);
    }
}

// NUEVA función: reproduce la animación de "Queja" una sola vez
void reproducirAnimacionQuejaUnaVez() {
    for (int i = 0; i < numFramesQueja; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesQueja[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayQueja);
    }
}

```

Esa función puede ir en cualquier parte del archivo siempre que esté fuera de otra función (a nivel global, como las demás), pero ponerla junto a su "hermana" (reproducirAnimacionUnaVez) hace el código más fácil de leer y mantener.

Usuario

Tengo un problema con la nueva animación, y es que me añade partes que no están

Claude

¿Podrías contarme un poco más específicamente qué ves? Por ejemplo:

1. ¿Aparecen puntos o motas dispersas por toda la pantalla, como "ruido" que no forma parte de la imagen final?
2. ¿Se ve texto o parte del verso anterior mezclado con la animación (como si no se hubiera borrado la pantalla)?
3. ¿La imagen final (frame 13) se ve distinta a la forma que esperabas, con partes de más en zonas donde no debería haber nada?

Esto importa porque, si es la opción 1, es probable que sea el comportamiento esperado del dithering: los bitmaps de animacionQueja.h fueron generados con el método "Ordered (Bayer)", que dibuja los píxeles oscuros como una nube de puntos dispersos que se va llenando progresivamente. Eso es intencional para dar un efecto de "materialización", pero si no es lo que esperabas visualmente, puede parecer un error.

Si es la opción 2, sí sería un bug de código (por ejemplo, si reproducirAnimacionQuejaUnaVez() no está limpiando bien la pantalla, o si display.clearDisplay() no se ejecuta donde debería).

¿Cuál de estas se parece más a lo que estás viendo? Si puedes, cuéntame también en qué posición del potenciómetro ocurre.

Usuario

Podemos intentar los 3?

Claude

Genial, vamos por partes. Aquí tienes 3 pruebas simples, una por cada hipótesis:

Prueba 1: ¿Es el dithering esperado?

Deja el potenciómetro fijo en la posición de la animación (posición 1) y observa toda la secuencia sin tocar nada. Debería verse como una nube de puntos sueltos que va aumentando hasta formar una imagen completa, y luego disminuyendo hasta quedar vacía otra vez (frame00→13→00). Si es justo eso, es el comportamiento normal del efecto Bayer, no un bug.

Prueba 2: ¿Queda texto pegado de antes?

Fíjate en el instante justo en que el verso "Alfonsina Storni" desaparece y arranca la animación. ¿Ves letras o fragmentos del texto anterior mezclados con los puntos de la animación? Si es así, es señal de que `clearDisplay()` no se está ejecutando donde debería.

Prueba 3: ¿La imagen final tiene partes de más?

Deja el potenciómetro quieto hasta que la animación termine en su frame final (frame 13). Compara esa imagen final con lo que esperabas ver. Si ves manchas o formas sin sentido, el problema podría estar en cómo se generó el bitmap.

Notas adicionales: el comentario `"// Image: quejaFrame13"` aparece en TODOS los frames, probablemente solo un detalle de la herramienta generadora. Además, `framesQueja[]` hace un efecto "ida y vuelta" (sube de 00 a 13 y baja de nuevo a 00), por lo que la animación dura el doble de lo esperado (28 frames en vez de 14).

Usuario

Creo que es la última

Claude

Buena pista. Vamos a "ver" realmente el frame 13 (el más completo) para comparar qué forma dibuja de verdad, decodificando los bytes y generándolo como imagen.

[Se decodificó el bitmap del frame 13 en dos órdenes de bits distintos: MSB-first y LSB-first, y se renderizaron como imágenes para comparar]

Resultado: decodificado en orden MSB-first (el que usa `Adafruit_GFX` normalmente), la imagen se ve como texto distorsionado y fragmentado. Decodificado en orden LSB-first (como indica el comentario del propio archivo .h), la imagen se ve perfectamente clara: dice la palabra "QUEJA".

Diagnóstico

¡Ahí está el problema! El bitmap en sí está bien — dice literalmente "QUEJA" al decodificarlo en el orden correcto (LSB-first). El problema es que `Adafruit_GFX::drawBitmap()` espera el orden MSB-first, no LSB-first. Por eso, cuando el Arduino dibuja esos mismos bytes, cada byte se interpreta "al revés" bit por bit, y el resultado es el texto distorsionado que se ve en pantalla — con pedazos que parecen de más porque las letras quedan "despedazadas" y recombinadas mal, no porque falte o sobre contenido real.

Cómo arreglarlo

Opción A — Re-generar el .h con el orden correcto (recomendado): si se usó una herramienta/conversor online para generar `animacionQueja.h`, probablemente tiene una opción "bit order" o "LSB/MSB first" — cambiarla a MSB first y volver a exportar.

Opción B — Invertir los bits en el propio Arduino, sin regenerar nada:

```
uint8_t invertirBits(uint8_t b) {  
    b = (b & 0xF0) >> 4 | (b & 0x0F) << 4;  
    b = (b & 0xCC) >> 2 | (b & 0x33) << 2;  
    b = (b & 0xAA) >> 1 | (b & 0x55) << 1;  
}
```

```

    return b;
}

void reproducirAnimacionQuejaUnaVez() {
    for (int i = 0; i < numFramesQueja; i++) {
        display.clearDisplay();
        for (int b = 0; b < 512; b++) {
            uint8_t original = pgm_read_byte(&framesQueja[i][b]);
            uint8_t invertido = invertirBits(original);
            int x = (b % 16) * 8;
            int y = b / 16;
            for (int bit = 0; bit < 8; bit++) {
                if (invertido & (0x80 >> bit)) {
                    display.drawPixel(x + bit, y, SSD1306_WHITE);
                }
            }
        }
        display.display();
        delay(frameDelayQueja);
    }
}

```

La Opción B es un poco más lenta (dibuja pixel por pixel), pero funciona sin tocar el .h. La Opción A es más simple y eficiente si se puede regenerar el archivo.

Usuario

Era la A, muchas graciasss

Claude

¡Genial, me alegra que se haya resuelto! Sí, mucho más simple regenerar con la opción correcta que andar invirtiendo bits a mano.

Si vuelve a pasar algo raro con otra animación (o con las llamas), ya se sabe: se puede decodificar el .h y renderizarlo como imagen para ver justo qué está mal, en vez de adivinar leyendo hexadecimales.

¡Que disfrutes viendo "Queja" cobrar vida en la pantallita!

Usuario

[se adjuntó una nueva versión del sketch, con una tercera animación ("Corazón") añadida, LED de "grito", y varios errores de compilación] y ahora me puedes añadir una nueva animación? es que me lanza un error y no entiendo como modificarlo.

Claude

¡Encontré el problema! Hay dos errores de compilación y un tercer bug "silencioso" que no da error pero rompería la sincronización de los versos.

Error 1: Llave de más en el loop()

El código tenía una llave "}" extra antes del "else", que cerraba el bloque if (posicionActual != posicionAnterior) antes de tiempo, provocando el error 'else' without a previous 'if'.

Solución: borrar esa llave suelta, para que el else sea parte de la misma cadena if / else if / else if / else:

```

if (posicionActual == posicionAnimacionQueja) {
    reproducirAnimacionQuejaUnaVez();

} else if (posicionActual == posicionAnimacionLlamas) {
    reproducirAnimacionLlamasUnaVez();

} else if (posicionActual == posicionAnimacionCorazon) {

```

```

        reproducirAnimacionCorazonUnaVez();

    } else {
        int indiceActual = posicionActual;
        ...
    }

```

Error 2: Nombre de función que no existe

Se llamaba a reproducirAnimacionLlamasUnaVez(), pero la función definida se llamaba reproducirAnimacionUnaVez() (sin "Llamas" en el nombre), dando el error 'reproducirAnimacionLlamasUnaVez' was not declared in this scope.

Solución recomendada: renombrar la función de llamas para que los tres nombres sean consistentes:

```

void reproducirAnimacionLlamasUnaVez() {
    for (int i = 0; i < numFramesLlamas; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesLlamas[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayLlamas);
    }
}

```

Error 3: Falta la función de la animación del corazón

Se llamaba a reproducirAnimacionCorazonUnaVez(), pero nunca se definió — solo se incluyó el .h con los frames. Se agrega junto a las otras dos, con el mismo patrón:

```

void reproducirAnimacionCorazonUnaVez() {
    for (int i = 0; i < numFramesCorazon; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesCorazon[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayCorazon);
    }
}

```

Bug silencioso: cálculo del índice del verso incompleto

Con 3 posiciones de animación (Queja, Corazón, Llamas), el cálculo del índice del verso solo restaba 1 por cada una de dos, faltando restar por el Corazón, lo que descuadraría los versos a partir de esa posición.

```

int indiceActual = posicionActual;
if (posicionActual > posicionAnimacionQueja) indiceActual--;
if (posicionActual > posicionAnimacionCorazon) indiceActual--;
if (posicionActual > posicionAnimacionLlamas) indiceActual--;

```

Escala final del potenciómetro (0 a 15, total 16 posiciones): 0 = Verso 0 · 1 = Animación Queja · 2–5 = Versos 1–4 · 6 = Animación Corazón · 7–9 = Versos 5–7 · 10 = Animación Llamas · 11–15 = Versos 8–12.

Con esos tres cambios (llave, nombre de función, y la línea que falta en el indiceActual) debería compilar y funcionar bien. Quedó pendiente revisar el LED_GRITO, ya que quedó dentro del bloque else que se movió — hay que confirmar que no haya quedado fuera de lugar tras el arreglo.

Usuario

Holaaa, puedes generar el pdf de esta conversación en pdf? por favorr

— Fin de la conversación exportada —